

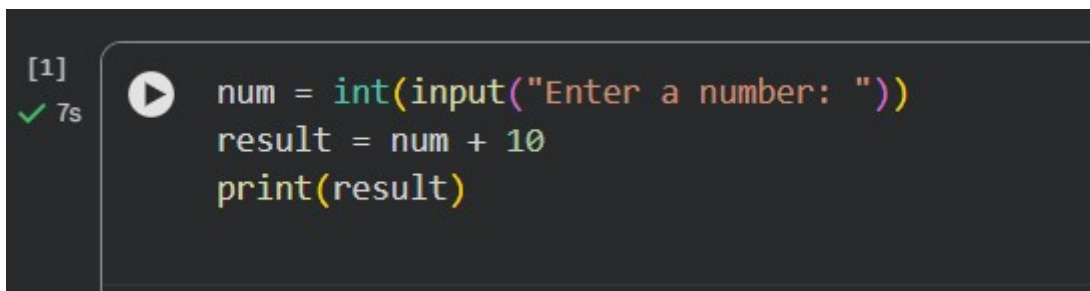
School of Computer Science and Artificial Intelligence

Lab Assignment # 7.2

Program	: B. Tech (CSE)
Specialization	:
Course Title	: AI Assisted coding
Course Code	:
Semester	: II
Academic Session	: 2025-2026
Name of Student	:B.Gnaneshwar
Enrollment No.	: 2403A51L15
Batch No.	: 51
Date	:30-01-2026

TASK 1: Runtime Error Due to Invalid Input Type**Prompt:**

Use AI tools to identify the runtime error caused by invalid input type and correct the program so that it executes properly.

Code :

```
[1] 7s num = int(input("Enter a number: "))
result = num + 10
print(result)
```

Explanation:

- `input ()` takes user input as a **string** by default.
- The program tries to add an integer (10) to a string (num).

- Python does not allow addition between a string and an integer.
- AI identifies the issue and suggests converting the input to an integer using `int()`.
- After conversion, arithmetic operation works correctly.

Output :

```
... Enter a number: 52
62
```

TASK 2: Incorrect Function Return Value

Prompt:

Use AI assistance to analyze the function and ensure that it correctly returns the computed value.

Code :

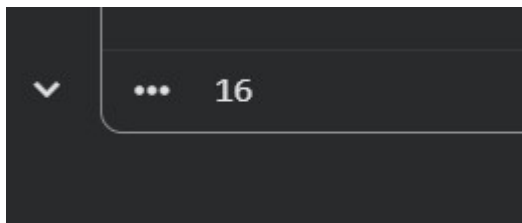
```
[2] ✓ 0s  def square(n):  
    return n * n  
  
print(square(4))
```

Explanation:

- The function calculates the square of a number.
- The calculated result is stored in a variable but not returned.

- Without a `return` statement, the function returns `None`.
- AI detects the missing return statement.
- Adding `return` allows the function to send the result back correctly.

Output :

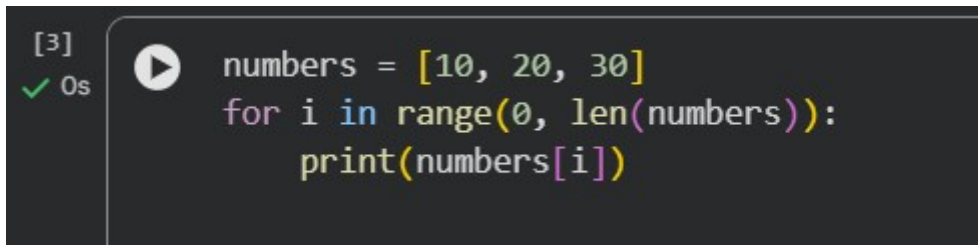


TASK 3: IndexError in List Traversal

Prompt:

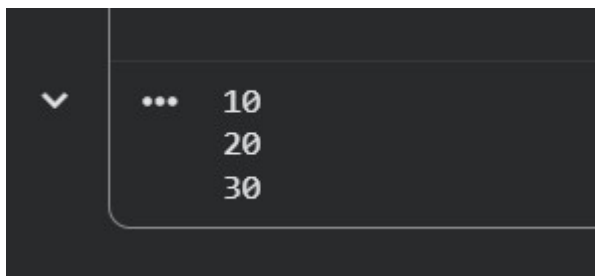
Use AI to identify the incorrect loop boundary and correct the iteration logic.

Code :

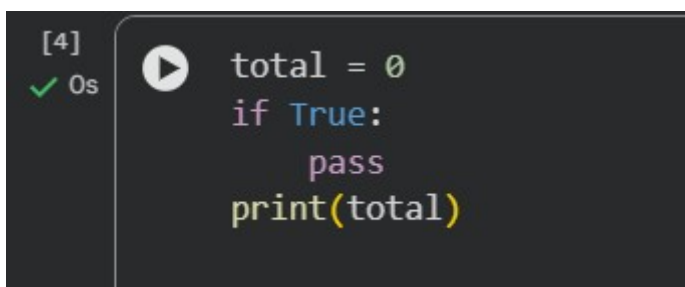


Explanation:

- The list contains three elements.
- `len(numbers)` gives the total count of elements.
- Using `len(numbers) + 1` causes the loop to access an invalid index.
- This results in an `IndexError`.
- AI suggests correcting the loop boundary to prevent out-of-range access.

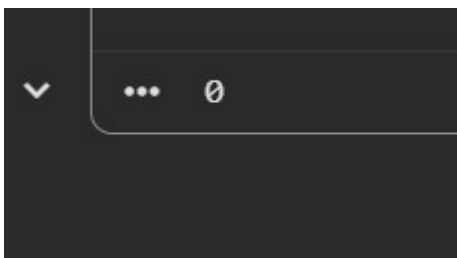
Output :**TASK 4: Uninitialized Variable Usage****Prompt:**

Use AI tools to detect the uninitialized variable and correct the program.

Code :

Explanation:

- The variable `total` is used before assigning any value.
- Python raises a `NameError` when an undefined variable is accessed.
- AI identifies the missing initialization.
- Initializing the variable before use fixes the error.

Output :**TASK 5: Logical Error in Student Grading System****Prompt:**

Use AI to analyze the grading conditions and correct the logical flow.

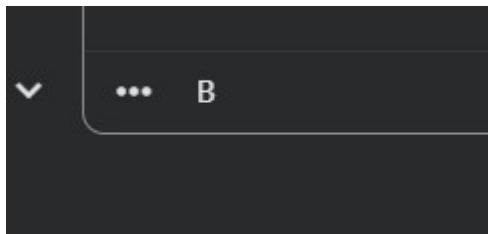
Code :

```
[5]
✓ 0s
marks = 85
if marks >= 90:
    grade = "A"
elif marks >= 80:
    grade = "B"
else:
    grade = "C"
print(grade)
```

Explanation:

- The program assigns grades based on marks.
- Marks between 80 and 89 are incorrectly assigned grade "C".
- Logical ordering of conditions is incorrect.
- AI identifies the logical error and rearranges the grading conditions.
- Correct logic assigns grades accurately.

Output :



Final Conclusion (5 Points)

- AI tools help in quickly identifying **syntax, runtime, and logical errors** in Python programs.
- AI provides **clear explanations** for errors, making debugging easier for beginners.
- Using AI improves **systematic debugging skills** and reduces trial-and-error coding.
- AI-assisted suggestions help in **writing cleaner and more reliable code**.
- This lab increased confidence in using AI as an effective **programming support tool**.